Mid-Term Examination – Python Programming

Total Marks: 40 **Duration:** 1.5 Hours

Topics: Loops, Strings, Lists, Tuples, Dictionaries Functions, NumPy, OOP, Git, Pandas

Section A: MCQs / Short Answer

1. **[Lists: 2 marks]** What will be the output of the following code snippet?

```
nums = [1, 2, 3]
nums.append([4, 5])
print(len(nums))
```

Answer: 4
Explanation: `append()` adds the list as a single element.

2. **[Functions: 2 marks]** What does the following code snippet print?

```
def f(x, y=5):
    return x + y
print(f(3)+f(3,2))
```

Answer: 13
Explanation: Default argument y=5 used for f(3), overridden in f(3,2) and final answer is $(3+5)+(3+2)=13$

3. **[MCQ: 2 marks]** What is the output of the following code snippet? Give your explanation with a mathematical expression.

```
def mystery(n):
    if n <= 1:
        return n
    else:
        return mystery(n - 1) + mystery(n - 2)

print(mystery(4))
```

A) 3 **B)** 4 **C)** 5 **D)** 6

Answer: A) 3
Explanation: mystery(4) = mystery(3) + mystery(2) = mystery(2) + mystery(1) + mystery(1) + mystery(0) = mystery(1) + mystery(0) + 1 + 1 + 0 = 1 + 0 + 2 = 3

4. **[NumPy Multiplication: 2 marks]** If a and b are two dimensional NumPy arrays with shapes $(2,3,4)$ and $(1,4)$. What will be shape of the output array when `a*b` is performed? If the operation results in an error, write the answer as "None", else write the shape in this form (x,y) or (x,y,z) without spaces inbetween.

Answer: (2,3,4)
Explanation: NumPy treats (1,4) as (1,1,4), broadcasts it to match (2,3,4) — the

leading 1 expands to 2, the middle 1 expands to 3, so elementwise multiplication works and the resulting shape is (2,3,4)

5. **[NumPy Stack: 2 marks]** What will be the resulting matrix of the following NumPy code snippet?

```
import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
print(np.vstack((a,b)))
```

A.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

B.

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \end{bmatrix}$$

C.

$$\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

D.

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Answer: A
Explanation: Vstack combined numpy arrays by placing them one below the other.

6. **[NumPy View vs Copy: 2 marks]** What will be the output of the following NumPy code snippet?

```
a = np.array([[1,2,3],[4,5,6]])
b = a[:, :2]
b[0,0] = 99
print(a[0,0])
```

Answer: 99
Explanation: Slice creates a view; modifying b modifies a.

7. **[OOP:2 marks]** What will be the output of the following code snippet?

```
class test:
    def __init__(self):
        self.variable = 'Old'
        self.Change(self.variable)
    def Change(self, var):
```

```
        var = 'New'
obj=test()
print(obj.variable)
```

A. new

B. old

C. error

D. nothing

Answer: B
Explanation: In the __init__ method, self.variable is set to 'Old'. Then, self.Change(self.variable) is called, passing the string 'Old' as an argument. Inside the Change method, the parameter var is assigned 'New', but this only changes the local variable var and does not affect self.variable. Therefore, when print(obj.variable) is called, it prints the original value 'Old'.

8. **[OOP:1 marks]** What does the __str__ method do in Python?

A. Compares two objects

B. Converts a string to an object

C. Converts an object to a string

D. Initializes an object

Answer: C

9. **[OOP:1 mark]** What type of attributes are unique to each individual object created from a given class?
Answer: Instance attributes (or) Instance (or) Instance attribute

10. **[OOP: 1 mark]** What does the *self* keyword represent in the methods of Python class?

A. It represents the superclass

B. It represents the subclass

C. It represents the object instance

D. It represents the class

Answer: C

11. **[Git: 1 mark]** Suppose you create a new branch `experiment` from `main`, make one commit on `experiment`, and another commit on `main`. Which of the following statements correctly describes Git's internal pointers?

A. Both branches point to the same commit until merged.

B. `HEAD` points to both branches simultaneously.

C. Each branch points to its latest commit; they share a common ancestor but diverged afterward.

D. Branch pointers store separate copies of the entire project files.

Correct answer: C

12. **[Git: 2 marks]** You have already pushed a faulty commit to a shared remote repository. You want to undo its effects without disturbing your teammates' copies of the repository. Which is the command to be used and what does it do?

    A. `git revert <commit-hash>` — Deletes the specified commit from history and moves the branch pointer backward to the previous commit.

    B. `git revert <commit-hash>` — Creates a new commit that inverses the changes made by the specified commit, keeping the original commit history intact.

    C. `git reset -hard HEAD~1` — Creates a new commit with the opposite changes of the last commit to safely undo its effect.

    D. `git reset -hard HEAD~1` — Moves the branch pointer backward and erases all changes introduced by the last commit, rewriting history.

    Answer: B

13. **[Pandas: 2 marks]** Consider the following dataframe:

```
import pandas as pd

df = pd.DataFrame({
    "SKU":   ["P101", "P102", "P205", "P301", "P410"],
    "Price": [1200,   1500,   1800,   900,    1100 ]
})
```

Suppose it is re-indexed by SKU. Your task is to increase `Price` by 5% for SKUs "P102" and "P205" and then display only those rows. The code must remain correct even if the DataFrame is later sorted by `Price` or has new rows inserted. Which of the following code snippets does it correctly ? Select multiple options if more than one answer is right.

A.

```
df.loc[["P102", "P205"], "Price"] *= 1.05
df.loc[["P102", "P205"], ["Price"]]
```

—

B.

```
df.loc["P102":"P205", "Price"] *= 1.05
df.loc["P102":"P205", ["Price"]]
```

—

C.

```
df.iloc[[1, 2], 0] *= 1.05
df.iloc[[1, 2], [0]]
```

D.

```
df.iloc[1:3, 0] *= 1.05
df.iloc[1:3, [0]]
```

Correct Answer: A

Explanation: Only code snippet which selects by stable label keys (index values) which remain unaffected by sorting or inserted rows.

Section B: Programming Based

1. **[Strings: 4 marks - 2 per test case]** Write a Python function `reverse_string(s: str)` that returns the reverse of a string **without using slicing**.

   Note: Using slicing will receive zero marks.

   Example:

   ```
   s = "Hello"
   print(reverse_string(s))
   # Expected Output: "olleH"
   ```

   Test Cases:

   ```
   import inspect

   # Automated check to ensure slicing is not used
   source_code = inspect.getsource(reverse_string)
   assert '[:' not in source_code and ':-' not in source_code, "0
      marks: Slicing used!"

   # Test Case 1: Normal string
   s1 = "Hello World"
   expected1 = "dlroW olleH"
   assert reverse_string(s1) == expected1, "Test Case 1 Failed"
   print("Test Case 1 Passed")

   # Test Case 2: Edge case - empty string
   s2 = ""
   expected2 = ""
   assert reverse_string(s2) == expected2, "Test Case 2 Failed"
   print("Test Case 2 Passed")
   ```

2. **[Functions: 4 marks - 2 per test case]** Define a function `count_vowels(s: str) -> int` that returns the number of vowels in a given string `s`. Vowels are `a, e, i, o, u` in both lowercase and uppercase.

   Test Cases:

```
# Test Case 1: Normal string with mixed case
s1 = "Education"
expected1 = 5
assert count_vowels(s1) == expected1, "Test Case 1 Failed"
print("Test Case 1 Passed")

# Test Case 2: Edge case - no vowels and empty string
s2 = "Rhythm"
expected2 = 0
assert count_vowels(s2) == expected2, "Test Case 2a Failed"
s3 = ""
expected3 = 0
assert count_vowels(s3) == expected3, "Test Case 2b Failed"
print("Test Case 2 Passed")
```

3. **[Functions + Lists: 4 marks - 2 per test case]** Write a function `find_max(lst: list) -> int` that returns the **maximum element** in a list **without using the built-in max() function**.

   Note: Using the built-in `max()` function will receive zero marks.

   Test Cases:

```
import inspect

# Automated check to ensure max() is not used
source_code = inspect.getsource(find_max)
assert "max(" not in source_code, "0 marks: max() function used!"

# Test Case 1: Normal list
lst1 = [4, 10, 2, 7]
expected1 = 10
assert find_max(lst1) == expected1, "Test Case 1 Failed"
print("Test Case 1 Passed")

# Test Case 2: Edge case - all negative numbers
lst2 = [-5, -10, -3, -8]
expected2 = -3
assert find_max(lst2) == expected2, "Test Case 2 Failed"
print("Test Case 2 Passed")
```

4. **[NumPy: 4 marks - 2 per test case]** Write a Python function `replace_odds(arr: np.ndarray) -> np.ndarray` that takes a NumPy array as input and replaces all odd numbers in the array with `-1`.

   Test Cases:

```
import numpy as np

# Test Case 1: 1 to 10
arr1 = np.arange(1, 11)
```

```
expected1 = np.array([-1, 2, -1, 4, -1, 6, -1, 8, -1, 10])
assert np.array_equal(replace_odds(arr1), expected1), "Test Case 1
    Failed"
print("Test Case 1 Passed")

# Test Case 2: Including negative numbers and zero
arr2 = np.array([0, 1, -3, 4, 5, -6])
expected2 = np.array([0, -1, -1, 4, -1, -6])
assert np.array_equal(replace_odds(arr2), expected2), "Test Case 2
    Failed"
print("Test Case 2 Passed")
```

5. **[NumPy: 4 marks - 2 per test case]** Normalize a 2D matrix X (min–max normalization).

   Problem: Write a function `two_d_norm(X)` that takes a 2D matrix `X` as input and normalizes it so that **all elements of the entire matrix** lie between 0 and 1 using global min–max normalization. That is, use the minimum and maximum values of the whole matrix in the formula:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

   Test Cases:

```
# Test Case 1
X1 = np.array([[10, 20, 30],
               [5, 15, 25],
               [0, 50, 100]])

expected_X1 = (X1 - X1.min()) / (X1.max() - X1.min())
assert np.allclose(two_d_norm(X1), expected_X1), "Test Case 1
   Failed"
print("Test Case 1 Passed")

# Test Case 2:
X2 = np.array([[-1000, 0, 1000],
               [5000, -500, 2000],
               [10000, 500, -2000]])

expected_X2 = (X2 - X2.min()) / (X2.max() - X2.min())
assert np.allclose(two_d_norm(X2), expected_X2), "Test Case 2
   Failed"
print("Test Case 2 Passed")
```

6. **[Python Classes: 6 marks - 2 per test case]** Implement a class `ShoppingCart` to manage a list of items, where each item is a tuple `(name: str, price: float)`.

   The class must include the following methods:

- `add_item(name: str, price: float) -> None`
  Adds an item to the cart.
- `remove_item(name: str) -> bool`
  Removes the **first** item with the given name. Returns `True` if removed, `False` if not found.
- `calculate_total() -> float`
  Returns the total price of all items in the cart.
- The methods should handle possible edge cases as well

Test Cases:

```
# Test Case 1: Basic add, remove, and calculate total
cart1 = ShoppingCart()
cart1.add_item("Papaya", 100)
cart1.add_item("Guava", 200)
cart1.add_item("Orange", 150)
assert cart1.calculate_total() == 450, "Test Case 1 Failed (Total)
    "
assert cart1.remove_item("Orange") == True, "Test Case 1 Failed (
    Remove existing item)"
assert cart1.calculate_total() == 300, "Test Case 1 Failed (Total
    after removal)"
assert cart1.remove_item("Orange") == False, "Test Case 1 Failed (
    Remove non-existing item)"
print("Test Case 1 Passed")

# Test Case 2: Multiple items with same name
cart2 = ShoppingCart()
cart2.add_item("Apple", 50)
cart2.add_item("Apple", 70)
assert cart2.calculate_total() == 120, "Test Case 2 Failed (Total)
    "
assert cart2.remove_item("Apple") == True, "Test Case 2 Failed (
    Remove first Apple)"
assert cart2.calculate_total() == 70, "Test Case 2 Failed (Total
    after removal)"
print("Test Case 2 Passed")

# Test Case 3: Edge case - empty cart
cart3 = ShoppingCart()
assert cart3.calculate_total() == 0, "Test Case 3 Failed (Empty
    cart total)"
assert cart3.remove_item("Banana") == False, "Test Case 3 Failed (
    Remove from empty cart)"
print("Test Case 3 Passed")
```

7. **[Lambda function: 6 marks - 3 per test case]** A company wants to compute discounts for online orders based on the **type of customer** and the **order amount**.

A company wants to compute discounts for online orders based on the **type of customer** and the **order amount**.

Write a Python function named `apply_discount(order_list)` that accepts a list of tuples, where each tuple contains the `(customer_type, order_amount)`.

The function should:

(a) Apply the following discount rules:
- **Prime customers**: 20% discount on all orders.
- **Regular customers**: 10% discount only if the order amount exceeds 1000.
- **Guest customers**: No discount.

(b) Note: There are only 3 possible customer types — `"prime"`, `"regular"`, and `"guest"`.

(c) Return a list of final payable amounts (after applying discounts) in the same order as the input.

Example:

```
orders = [("regular", 1200.0), ("prime", 800.0), ("guest",
1500.0)]
print(apply_discount(orders))
# Expected Output: [1080.0, 640.0, 1500.0]
```

Note: Submissions that do not use a lambda function will receive **zero marks**, even if the output is correct.

Test Cases:

```
import inspect

# Verify that a lambda function is used
source_code = inspect.getsource(apply_discount)
assert "lambda" in source_code, "Test Failed: No lambda function
   used!"
print("Lambda usage verified.")

# Test Case 1:
orders1 = [
    ("regular", 1200.0), ("prime", 800.0), ("guest", 1500.0),
    ("prime", 2000.0), ("regular", 900.0)
]
expected1 = [1080.0, 640.0, 1500.0, 1600.0, 900.0]
assert apply_discount(orders1) == expected1, "Test Case 1 Failed"
print("Test Case 1 Passed")

# Test Case 2:
orders2 = [
    ("prime", 500.0), ("prime", 1500.0), ("regular", 2000.0),
    ("regular", 800.0), ("guest", 3000.0)
]
expected2 = [400.0, 1200.0, 1800.0, 800.0, 3000.0]
assert apply_discount(orders2) == expected2, "Test Case 2 Failed"
print("Test Case 2 Passed")
```

```
# Test Case 3:
orders3 = [
    ("prime", 100000.0), ("regular", 1001.0), ("guest", 99999.0),
    ("regular", 999.0), ("prime", 50.0)
]
expected3 = [80000.0, 900.9, 99999.0, 999.0, 40.0]
assert apply_discount(orders3) == expected3, "Test Case 3 Failed"
print("Test Case 3 Passed")
```

8. **[Pandas: 6 marks - 3 per test case]** You are given a sales dataset in the form of a Python dictionary. Here is a sample dataset for your reference:

```
sales_data = {
    "Region": ["North", "South", "East", "West", "North",
    "East", "South", "West"],
    "Product": ["A", "A", "B", "B", "C", "C", "A", "B"],
    "Sales": [1200, 800, 600, 900, 1500, 700, 650, 1100],
    "Month": ["Jan", "Jan", "Feb", "Feb", "Jan", "Feb",
    "Feb", "Jan"]
}
```

You must write a Python function named `group_sales_total()` that satisfies the following requirements:

(a) The function should accept four arguments:

- `data_dict` — the dictionary containing the data
- `month_filter` — a string specifying the month to filter (e.g. `"Jan"` or `"Feb"`)
- `group_var` — the column name to group by, either `"Region"` or `"Product"`
- `group_value` — a specific value from the above selected column for which the total sales are to be returned

(b) The function should:

i. Convert the dictionary into a tabular form suitable for efficient filtering and aggregation.

ii. Filter the data for the given `month_filter`.

iii. Compute total sales for each group (by `"Region"` or `"Product"`).

iv. Return the total sales corresponding to the given `group_value`.

v. Handle edge cases of group value not being present by returning 0

(c) The function should return a single numeric value representing the total sales of the specified group in the given month.

Example:

```
result = group_sales_total(sales_data, month_filter="Jan",
group_var = "Region", group_value="East")
print(result)
# Expected Output: 600
```

Test Cases:

```
# Test Case 1: Basic case (group by Region)
sales_data = {
    "Region": ["North", "South", "East", "West", "North", "East",
       "South", "West"],
    "Product": ["A", "A", "B", "B", "C", "C", "A", "B"],
    "Sales": [1200, 800, 600, 900, 1500, 700, 650, 1100],
    "Month": ["Jan", "Jan", "Feb", "Feb", "Jan", "Feb", "Feb", "
       Jan"]
}
result1 = group_sales_total(sales_data, month_filter="Jan", group_
   var="Region", group_value="West")
expected1 = 1100
assert result1 == expected1, "Test Case 1 Failed"
print("Test Case 1 Passed")

# Test Case 2: Group by Product for February
result2 = group_sales_total(sales_data, month_filter="Feb", group_
   var="Product", group_value="C")
expected2 = 700
assert result2 == expected2, "Test Case 2 Failed"
print("Test Case 2 Passed")

# Test Case 3: Edge case - group value not present in filtered
   data
result3 = group_sales_total(sales_data, month_filter="Jan", group_
   var="Region", group_value="Central")
expected3 = 0  # Should return 0 since 'Central' is not in the
   data
assert result3 == expected3, "Test Case 3 Failed"
print("Test Case 3 Passed")
```